FACULTY OF ARTS

KU LEUVEN

Scripting languages

Lecture 3: Conditions and repetition

Tim Van de Cruys

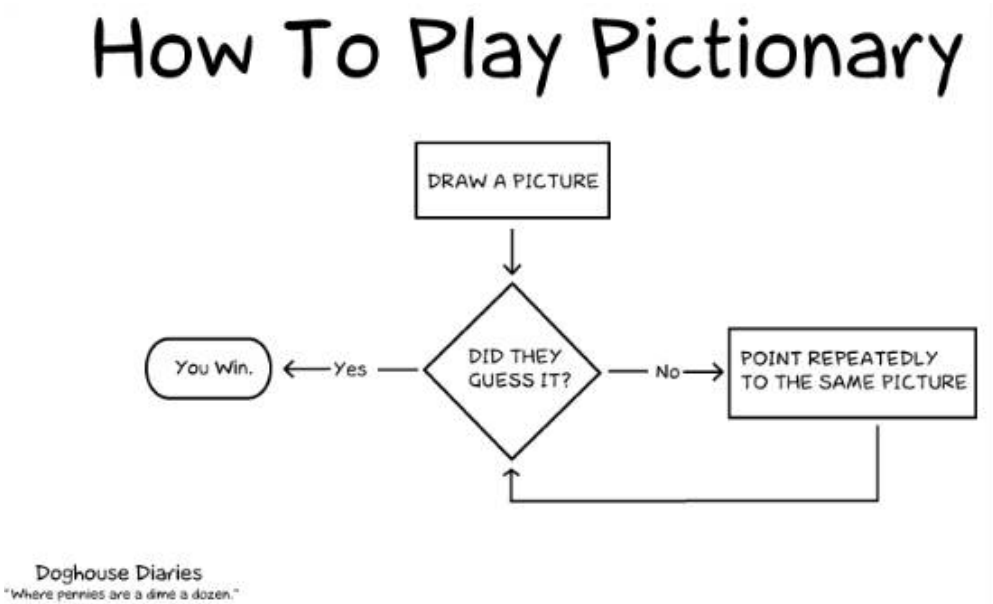


Wednesday 7 October 2026

Today

1. Conditions
   - Conditial expressions
   - Conditional execution: `if`-statement
2. Repetition
   - `for`-loop
   - `while`-statement
3. List comprehensions
4. Some example problems

Making decisions: flow charts



Simple conditions

- A **conditional expression** is an expression that is `True` or `False`
- Conditional operators:
  - `a == b`: `a` is equal to `b`
  - `a != b`: `a` is not equal to `b`
  - `a < b`: `a` is less than `b`?
  - `a > b`: `a` is greater than `b`?
  - `a <= b`: `a` is less than or equal to `b`?
  - `a >= b`: `a` is greater than or equal to `b`?
  - `a in b`: element `a` occurs in the sequence `b`
- Values and conditions that are `True` or `False` are called Boolean (George Boole, 1815–1864)
- Note: two equal signs ('==') tests equality, while one equal sign ('=') is an assignment statement!

Practical application: testing code

- Conditions can be used to verify that your code works as expected:

```
In: x = 3 * 3
In: print(x == 3 + 3 + 3)
True
```

- Useful debugging strategy: include checks throughout your code to test whether code is doing what it should do

The if-statement

```
if score >= 10:
    print('you passed the exam')
```

- The code under an `if`-statement is only executed if the condition is `True`
- The indentation determines which code is covered by the condition
- A **code block** is a sequence of lines after a statement ending in ':'
- The start and end of the block are defined by the amount of indentation (i.e., leading spaces) of the lines

if-statement examples

```
In: score = 11
In: if score >= 10:
In:     print('you passed the exam')
```

if-statement examples

```
In: score = 11
In: if score >= 10:
In:     print('you passed the exam')
```

```
In: score = 7
In: if score >= 10:
In:     print('you passed the exam')
```

if-statement examples

```
In: score = 11
In: if score >= 10:
In:     print('you passed the exam')
```

```
In: score = 7
In: if score >= 10:
In:     print('you passed the exam')
```

```
In: score = '12'
In: if score >= 10:
In:     print('you passed the exam')
```

Optional extensions of the if-statement

```
if score >= 10:
    print('you passed the exam')
else:
    print('you should have studied more')
```

- The code under `else` is executed if the condition does *not* match
- Condition must be either true or false, so one of the options will necessarily be executed

Optional extensions of the if-statement

```
if score > 16:
    print('you did really well!')
elif score >= 10:
    print('you passed the exam')
else:
    print('you should have studied more')
```

- It is possible to add multiple extra conditions with `elif` (else if)
- The conditions are tested from top to bottom; if one of them matches, the remainder of the conditions are ignored

Complex conditions

It is possible to create complex conditions from simple conditions:

- `cond1 and cond2` → `True` if both conditions are true
- `cond1 or cond2` → `True` if either of the conditions (or both) is true
- `not cond` → `True` if condition is `False`, and vice versa

For example:

```
if score >= 10 and score < 15:
    ...
```

- `and`, `or`, `not` are called **logical operators**

What counts as True and False?

- The number zero and empty sequences are treated as `False`
- Nonzero numbers and non-empty sequences are treated as `True`

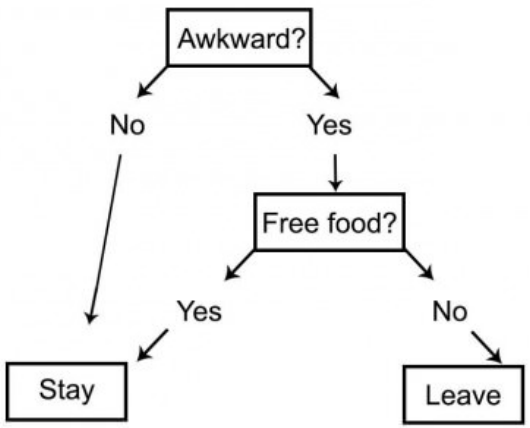
These are equivalent:

```
if len(shoppinglist) != 0:
    print('go to the supermarket')
if len(shoppinglist):
    print('go to the supermarket')
if shoppinglist:
    print('go to the supermarket')
```

In Python, we like to keep things simple: last one is preferred

Converting a decision tree into if-statements

Should you stay at the student reception?




```
awkward = False
free_food = True
```

Repeating statements: for-loops

```
shoppinglist = ['a', 'b', 'c']
for item in shoppinglist:
    print(item)
```

Result:

```
a
b
c
```

Repeating statements: for-loops

```
shoppinglist = ['a', 'b', 'c']
for item in shoppinglist:
    print(item)
```

Result:

```
a
b
c
```

The loop is equivalent to:

```
item = shoppinglist[0]
print(item)
item = shoppinglist[1]
print(item)
item = shoppinglist[2]
print(item)
```

- The statements within the for-loop are repeated for every element in the sequence
- At every iteration, `item` is assigned an element of `shoppinglist`

Looping over sequences

`for` loops can be used for anything with different objects (iterable):

```
for letter in 'word':
    print(letter)
```

Result:

```
w
o
r
d
```

```
for number in [0, 1, 2]:
    print(number)
```

Result:

```
0
1
2
```

Nesting loops inside loops

For loops can contain other for loops (nesting):

```
for number in [1, 2]:
    for letter in 'abc':
        print(number, letter)
```

What does the result look like?

Looping without a list

If we don't have a list but we want to repeat something a number of times, we can use the `range` function:

```
for number in range(4):
    print(number)
```

Result:

```
0
1
2
3
```

Repeats 4 times. Last item is 3 because counting starts at 0.

While-loop

```
countdown = 10
while countdown > 0:
    print(countdown)
    countdown -= 1
print('Liftoff!')
```

1. Evaluate the condition of the while-loop, which yields `True` or `False`
2. If the condition is false, exit the while statement and continue execution at the next statement
3. If the condition is true, execute the body and then go back to step 1

List comprehensions

- Specific kind of construction to create a new list from an existing sequence
- Makes use of a for loop

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
fruits_with_letter_a = []
for x in fruits:
    if "a" in x:
        fruits_with_letter_a.append(x)
```

List comprehensions

- Specific kind of construction to create a new list from an existing sequence
- Makes use of a for loop

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
fruits_with_letter_a = []
for x in fruits:
    if "a" in x:
        fruits_with_letter_a.append(x)
```

```
fruits_with_letter_a = [x for x in fruits if "a" in x]
```

List comprehensions

```
In: [i for i in range(10)]
```

List comprehensions

```
In: [i for i in range(10)]
```

```
In: [i for i in range(10) if i < 5]
```

List comprehensions

```
In: [i for i in range(10)]

In: [i for i in range(10) if i < 5]

In: [i for i in range(10) if i % 2 == 0]
```

List comprehensions

```
In: [i for i in range(10)]

In: [i for i in range(10) if i < 5]

In: [i for i in range(10) if i % 2 == 0]

In: [i*2 for i in range(10) if i >=5]
```

Example

How to count the number of ‘a’s in a string?

```
word = 'banana'
count = 0
```

Reversing text

Write code that reverses a string

For example, for the string `'I am testing'` you should get back the result `'gnitset ma I'`

```
string = 'I am testing'
result = ''
```

Palindromes

Write code that recognizes word palindromes (i.e. words that read the same written backwards and forwards)

For example, `'radar'` is a palindrome, but `'radio'` is not

```
string = 'radar'
```

Overlapping lists?

Write code that takes two lists and checks if they have at least one member in common

```
list1 = [1, 2, 3]
list2 = [1, 4, 6]
overlapping = False
```

Practical session / reading

- Practical session 2
- The code-along notebook of this lecture is on Toledo: re-run the examples at home
- Read chapters 5 & 6 of Python for Everybody